

Failure (prove the suite catches a regression) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `regression_catch_practice.py`: a green test suite only means something if you've seen it turn red for a real reason. Deleting one key instruction on purpose, and watching the score drop, is the proof. **If not:** the Build Task's regression check would be the first time you ever made a prompt worse on purpose — and you wouldn't know whether a missing drop meant "the prompt change was harmless" or "the suite is blind".

All versions use this prompt file — create it in `practice/` first:

```
# practice/support_prompt.txt
You are a support assistant for an online shop.
Always mention our 30-day return policy when a customer asks
about returns, refunds or exchanges.
Keep every reply under 60 words.
```

Every version calls the real OpenAI API, and runs from inside `practice/` with `python regression_catch_practice.py`. Read both depths — they're not "wrong, right," they're 2 real, valid ways to run the same check, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/regression_catch_practice.py
from openai import OpenAI

client = OpenAI()

TASKS = [
    {"id": "return_shoes",
     "question": "Can I return shoes I bought last week?",
     "must_include": "30"},
    {"id": "refund_lamp",
     "question": "How do I get a refund for a broken lamp?",
```

```

        "must_include": "30"},
    {"id": "exchange_shirt",
     "question": "Can I exchange a shirt for a bigger size?",
     "must_include": "30"},
    {"id": "send_back_jacket",
     "question": "I want to send back a jacket. What are the rules?",
     "must_include": "30"},
]

def load_prompt():
    with open("support_prompt.txt") as f:
        return f.read()

def answer(question):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": load_prompt()},
            {"role": "user", "content": question},
        ],
    )
    return response.choices[0].message.content

def run_eval_suite():
    results = {}
    for task in TASKS:
        reply = answer(task["question"])
        results[task["id"]] = task["must_include"] in reply
    return results

def count_passed(results):
    passed = 0
    for task_id in results:
        if results[task_id]:
            passed = passed + 1
    return passed

baseline = run_eval_suite()
print("baseline passed:", count_passed(baseline), "of", len(baseline))

input("Delete the 30-day line from support_prompt.txt, save, ")

```

```
        "then press Enter...")

after = run_eval_suite()
print("after sabotage passed:", count_passed(after), "of", len(after))

if count_passed(after) < count_passed(baseline):
    print("Suite works: it caught the regression.")
else:
    print("Suite has a hole: the score didn't drop.")
```

Expected output (real numbers can vary a little):

```
baseline passed: 4 of 4
Delete the 30-day line from support_prompt.txt, save, then press Enter...
after sabotage passed: 0 of 4
Suite works: it caught the regression.
```

Put the deleted line back in `support_prompt.txt` when you're done. This version reports *that* the score dropped, but not *which* task caught it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — overall score, with a fixed bar decided up front

Story: "the score dropped" can mean one task out of four wobbled. Deciding the bar *before* the sabotage — how many passes count as OK — stops you from explaining the result away afterwards. **If not:** any small dip could be read as "caught it", and a real hole in the suite could be read as noise.

```
# practice/regression_catch_practice.py
from openai import OpenAI

client = OpenAI()

# why: decided BEFORE sabotaging, so the result can't be
# explained away afterwards
PASS_BAR = 0.75

TASKS = [
    {"id": "return_shoes",
     "question": "Can I return shoes I bought last week?",
     "must_include": "30"},
    {"id": "refund_lamp",
     "question": "How do I get a refund for a broken lamp?",
     "must_include": "30"},
    {"id": "exchange_shirt",
     "question": "Can I exchange a shirt for a bigger size?",
     "must_include": "30"},
    {"id": "send_back_jacket",
     "question": "I want to send back a jacket. What are the rules?",
     "must_include": "30"},
]

def load_prompt() -> str:
    # how: read fresh on every call, so an edit to the file shows up
    # on the very next run without restarting the script
    with open("support_prompt.txt") as f:
        return f.read()
```

```

def answer(question: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": load_prompt()},
            {"role": "user", "content": question},
        ],
    )
    return response.choices[0].message.content

def run_eval_suite() -> dict:
    results = {}
    for task in TASKS:
        reply = answer(task["question"])
        results[task["id"]] = task["must_include"] in reply
    return results

def score(results: dict) -> float:
    passed = 0
    for task_id in results:
        if results[task_id]:
            passed = passed + 1
    return passed / len(results)

def run_regression_check() -> None:
    baseline = run_eval_suite()
    print(f"baseline score: {score(baseline):.2f}")

    input("Delete the 30-day line from support_prompt.txt, save, "
          "then press Enter...")

    after = run_eval_suite()
    print(f"after sabotage score: {score(after):.2f}")

    if score(baseline) >= PASS_BAR and score(after) < PASS_BAR:
        print("PASS: the suite caught the regression.")
    else:
        print("FAIL: the suite did NOT catch it. Tighten the rules.")

if __name__ == "__main__":
    run_regression_check()

```

Expected output:

```
baseline score: 1.00
Delete the 30-day line from support_prompt.txt, save, then press Enter...
after sabotage score: 0.00
PASS: the suite caught the regression.
```

Approach 2 — per-task comparison, to see exactly which task caught it

Story: when the check fails, "the score didn't drop" doesn't tell you where to look. Listing the tasks that passed before and fail after points straight at the rule that caught it — or, when the list is empty, at the task that *should* have. **If not:** a failed check would leave you guessing across the whole suite.

```
# practice/regression_catch_practice.py
from openai import OpenAI

client = OpenAI()

TASKS = [
    {"id": "return_shoes",
     "question": "Can I return shoes I bought last week?",
     "must_include": "30"},
    {"id": "refund_lamp",
     "question": "How do I get a refund for a broken lamp?",
     "must_include": "30"},
    {"id": "exchange_shirt",
     "question": "Can I exchange a shirt for a bigger size?",
     "must_include": "30"},
    {"id": "send_back_jacket",
     "question": "I want to send back a jacket. What are the rules?",
     "must_include": "30"},
]

def load_prompt() -> str:
    with open("support_prompt.txt") as f:
        return f.read()

def answer(question: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": load_prompt()},
```

```

        {"role": "user", "content": question},
    ],
)
return response.choices[0].message.content

def run_eval_suite() -> dict:
    results = {}
    for task in TASKS:
        reply = answer(task["question"])
        results[task["id"]] = task["must_include"] in reply
    return results

def find_newly_failed(baseline: dict, after: dict) -> list:
    newly_failed = []
    for task_id in baseline:
        # why: only a task that WAS passing counts as "caught" —
        # one that always failed tells you nothing about this change
        if baseline[task_id] and not after[task_id]:
            newly_failed.append(task_id)
    return newly_failed

def run_regression_check() -> None:
    baseline = run_eval_suite()
    input("Delete the 30-day line from support_prompt.txt, save, "
          "then press Enter...")
    after = run_eval_suite()

    newly_failed = find_newly_failed(baseline, after)
    if len(newly_failed) > 0:
        print("PASS: suite caught it. Newly failing:", newly_failed)
    else:
        print("FAIL: no task newly failed. Find the task that SHOULD "
              "cover this instruction, and tighten its rule.")

if __name__ == "__main__":
    run_regression_check()

```

Expected output (shown wrapped onto 2 lines just to fit the page — really one line of output):

```

Delete the 30-day line from support_prompt.txt, save, then press Enter...
PASS: suite caught it. Newly failing: ['return_shoes', 'refund_lamp',
'exchange_shirt', 'send_back_jacket']

```

Difference from Basic: Approach 1 turns "did the number go down" into a clear rule decided up front: the baseline must clear the bar and the sabotaged run must fall below it. Approach 2 keeps the per-task results and lists exactly which tasks went from pass to fail, so a failed check points straight at the rule to tighten.

Which one should you actually write? Approach 2 — the per-task list is what you need the moment the check fails, which is exactly when it matters. Approach 1's pass bar is worth adding on top once your suite has more tasks and one stray failure shouldn't count as "caught". The Build Task's `regression_check.py` uses both.